

Practical 1: Dynamic Memory Allocation

```
GNU nano 7.2 memory.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    // malloc
    int *a = (int *)malloc(3 * sizeof(int));

    printf("Using malloc:\n");
    for(i = 0; i < 3; i++)
    {
        a[i] = (i + 1) * 10;
        printf("%d ", a[i]);
    }

    // calloc
    int *b = (int *)calloc(3, sizeof(int));

    printf("\n\nUsing calloc:\n");
    for(i = 0; i < 3; i++)
        printf("%d ", b[i]);

    // realloc
    a = (int *)realloc(a, 5 * sizeof(int));

    printf("\n\nUsing realloc:\n");
    for(i = 3; i < 5; i++)
        a[i] = (i + 1) * 10;

    for(i = 0; i < 5; i++)
        printf("%d ", a[i]);

    // free
    free(a);
    free(b);

    printf("\n\nMemory freed successfully.\n");

    return 0;
}
```

```
saireddy@DESKTOP-HKQRL9G:~$ mkdir Dynamic_Memory
saireddy@DESKTOP-HKQRL9G:~$ cd Dynamic_Memory
saireddy@DESKTOP-HKQRL9G:~/Dynamic_Memory$ nano memory.c
saireddy@DESKTOP-HKQRL9G:~/Dynamic_Memory$ gcc memory.c -o memory
saireddy@DESKTOP-HKQRL9G:~/Dynamic_Memory$ ./memory
Using malloc:
10 20 30

Using calloc:
0 0 0

Using realloc:
10 20 30 40 50

Memory freed successfully.
saireddy@DESKTOP-HKQRL9G:~/Dynamic_Memory$ |
```

Practical 2: Copy-on-Write using fork()

```
GNU nano 7.2 copy.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int *data = malloc(sizeof(int));

    *data = 100;

    printf("Before fork:\n");
    printf("Parent PID: %d\n", getpid());
    printf("Data address: %p\n", (void *)data);
    printf("Data value: %d\n", *data);

    pid_t pid = fork();

    if(pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if(pid == 0)
    {
        printf("Child process:\n");
        printf("Child PID: %d\n", getpid());
        printf("Data address: %p\n", (void *)data);

        *data = 200;

        printf("Child modified data to: %d\n", *data);
        printf("Child data address: %p\n", (void *)data);
    }
    else
    {
        wait(NULL);

        printf("Parent process:\n");
        printf("Parent PID: %d\n", getpid());
        printf("Data value: %d\n", *data);
        printf("Data address: %p\n", (void *)data);

        free(data);

        return 0;
    }
}
```

```
Memory freed successfully.
saireddy@DESKTOP-HKQRL9G:~/Dynamic_Memory$ ^C
saireddy@DESKTOP-HKQRL9G:~/Dynamic_Memory$ cd ..
saireddy@DESKTOP-HKQRL9G:~$ mkdir COW_Practical
cd COW_Practical
saireddy@DESKTOP-HKQRL9G:~/COW_Practical$ nano cow.c
saireddy@DESKTOP-HKQRL9G:~/COW_Practical$ gcc cow.c -o cow
saireddy@DESKTOP-HKQRL9G:~/COW_Practical$ ./cow
Before fork:
Parent PID: 1733
Data address: 0x599276e1a2a0
Data value: 100

Child process:
Child PID: 1734
Data address: 0x599276e1a2a0
Child modified data to: 200
Child data address: 0x599276e1a2a0

Parent process:
Parent PID: 1733
Data value: 100
Data address: 0x599276e1a2a0
saireddy@DESKTOP-HKQRL9G:~/COW_Practical$ |
```